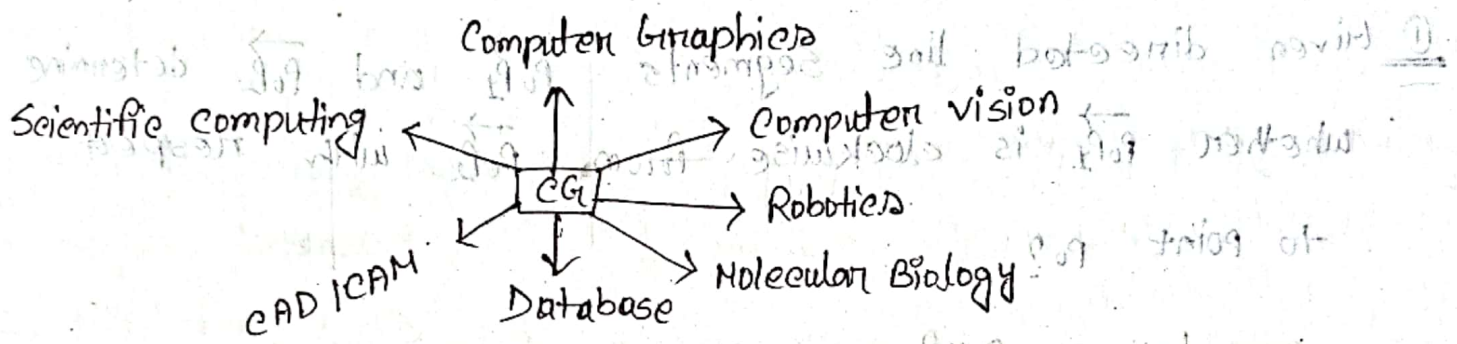


Segment-7 Computational Geometry

⇒ computational geometry is the branch of computer science that studies algorithms for solving geometric problems.
Deals with discrete shapes: points, lines, polyhedra, polygonal, meshes.



Application: ⇒ Occlusion, visibility, argumental reality
⇒ Collision detection, motion or assembly planning
⇒ Drug design, database, GIS, layout, fluid dynamics, etc.

Line segment: convex combination of two distinct points,
 $P_1 = (x_1, y_1)$
 $P_2 = (x_2, y_2)$ is any point $P_3 = (x_3, y_3)$ & α in the range

$$0 \leq \alpha \leq 1,$$

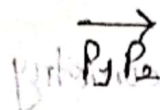
we have, $x_3 = \alpha x_1 + (1 - \alpha)x_2$

$$y_3 = \alpha y_1 + (1 - \alpha)y_2$$

$$P_3 = \alpha P_1 + (1 - \alpha)P_2$$

P_1, P_2 is the endpoints of segment $\overline{P_1 P_2}$

Directed segment



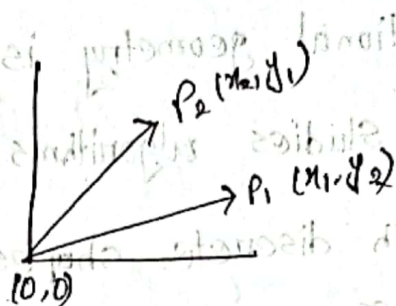
F. Example

Cross product:

$$U \times V = UV \sin \theta$$

$$P_1 \times P_2 = x_1 y_2 - x_2 y_1$$

$$= -P_2 \times P_1$$



① Given directed line segments $\vec{P_0 P_1}$ and $\vec{P_0 P_2}$ determine whether $\vec{P_0 P_1}$ is clockwise from $\vec{P_0 P_2}$ with respect to point P_0 ?

compute $\pi = P_1 \times P_2$

if $\pi > 0$; $\vec{P_0 P_1}$ is clockwise from $\vec{P_0 P_2}$

if $\pi < 0$; $\vec{P_0 P_1}$ is counterclockwise from $\vec{P_0 P_2}$

if $\pi > 0$; $\vec{P_0 P_2}$ is counterclockwise from $\vec{P_0 P_1}$

if $\pi < 0$; $\vec{P_0 P_2}$ is clockwise from $\vec{P_0 P_1}$

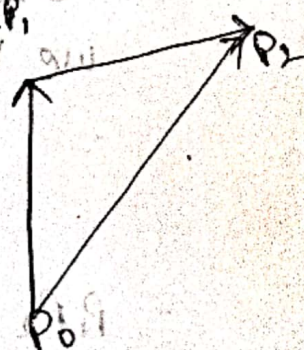
② Given two line segments $\vec{P_0 P_1}$ and $\vec{P_1 P_2}$ if we traverse $\vec{P_0 P_1}$ and then $\vec{P_1 P_2}$, do we make a left

turn at point P_1 ?

compute $\pi = (P_2 - P_0) \times (P_1 - P_0)$

if $\pi > 0$, then we make a right turn at P_1

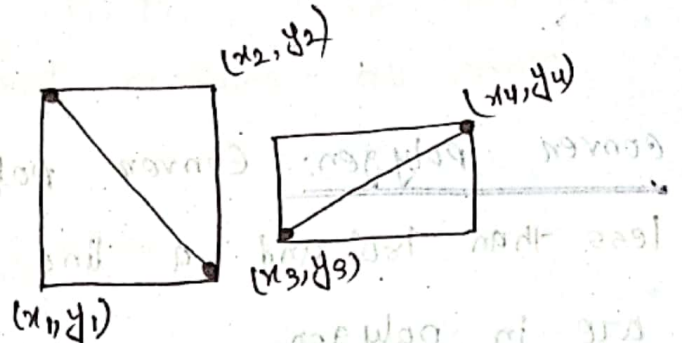
if $\pi < 0$, then we make a left turn at P_1



③ If two segments Intersect?

First draw a bound box

Case 1: Bounding boxes do not intersect neither will the segment.



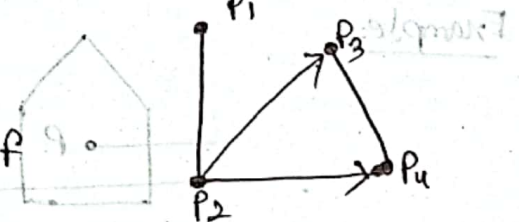
Bounding boxes do not intersect if and only if,
 $x_4 < x_1 \vee x_3 > x_2 \vee y_4 < y_1 \vee y_3 > y_2$

Case 2: Bounding boxes intersect, the segment may or maynot intersect.

Need further check:

① Line segments do not intersect if and only if one segment lines entirely to one side of the line containing the other segment.

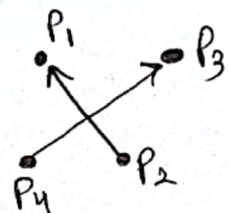
$(P_3 - P_2) \times (P_1 - P_2)$ and $(P_4 - P_2) \times (P_1 - P_2)$ are both positive or negative



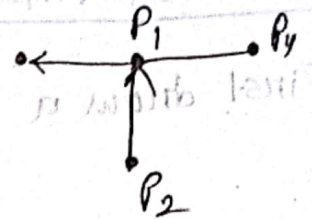
② Two line segments intersect if each of the two pairs of cross

Products have different signs (on one cross product in the pair is 0)

$(P_1 - P_4) \times (P_3 - P_4)$ and $(P_2 - P_4) \times (P_3 - P_4) \rightarrow$



$$(P_3 - P_2) \times (P_1 - P_2) \text{ and } (P_4 - P_2) \times (P_1 - P_2) \rightarrow$$



Convex polygon: Convex polygon has every interior angle less than 180° and a line between any two points are in polygon.

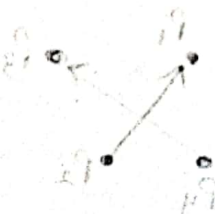
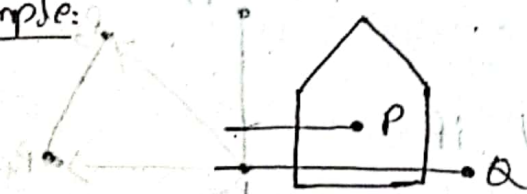
$\Rightarrow$ Non convex polygon may have one or more interior angles greater than 180° .

$\Rightarrow$ To determine whether a given point lies outside or inside a given polygon?

$\Rightarrow$ Draw a line along the x-axis from the given point in any direction.

$\Rightarrow$ If the line intersects only once with any edge of the polygon then the point lies inside the polygon else it lies outside the polygon.

Example:



Convex Hull problem:

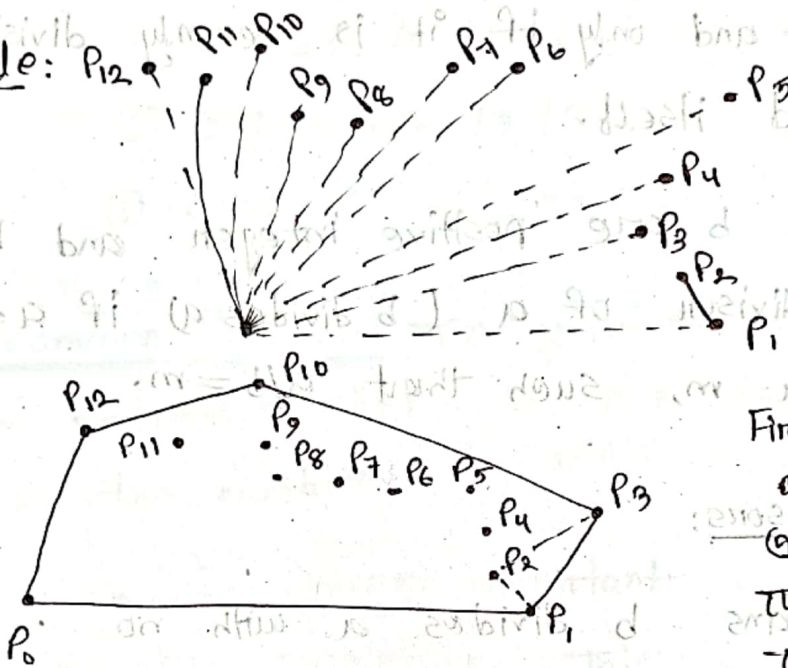
- Given n points on plane $P_1, P_2, \dots, P_n$ find the smallest convex polygon that contains all points.

$P_1, P_2, \dots, P_n$.

This problem is solved by Graham Scan:

Example: $P_{12}, P_{11}, P_{10}, P_9, P_8, P_7, P_6, P_5, P_4, P_3, P_2, P_1$

Ans:



First $P_1 P_1$ and $P_1 P_2$

are chosen as the initial edge.

Then the points are sorted by their polar angle with respect to P_1 .

Then the points are scanned in order, and the convex hull is built by adding points that make a left turn.

Points that make a right turn are rejected.

The final convex hull is the polygon formed by the points that remain.

The complexity of the algorithm is $O(n \log n)$.

Answer.

Complexity: $O(n \log n)$

Number theory:

Number used in Cryptography:

Non-negative numbers: integers

Prime: A positive integer greater than 1 is prime if and only if it is evenly divisible by only 1 and itself.

Divisors: If a and b are positive integers and $b \neq 0$, b is a divisor of a (b divides a) if $a = mb$ for some integer m , such that $a/b = m$.

Properties of divisors:

Notation: $b|a$ means b divides a with no remainder or b is a divisor of a .

If $a|k$ and $a|l$ then $a|(mk + nl)$ for arbitrary m and n .

Example: If $a = 6$; $k = 36$; $l = 54$, $m = 2$, $n = 3$

So, $6|36 = 6$ and $6|54$ and so does $6|(2 \times 36 + 3 \times 54)$

Division theorem: Let a, b and c be the integers.

Then:

$$\textcircled{1} a|b \wedge a|c \rightarrow a|(b+c)$$

$$\textcircled{2} a|b \rightarrow a|bc$$

$$\textcircled{3} a|b \wedge b|c \rightarrow a|c$$

Example: $\textcircled{1} 17|34 \wedge 17|170 \rightarrow 17|204$

$$\textcircled{2} 17|34 \rightarrow 17|340$$

$$\textcircled{3} 6|12 \wedge 12|144 \rightarrow 6|144$$

Greatest common divisor: The greatest common divisor (gcd) of two natural number is the largest division shared by these two numbers.

Modular Arithmetic: Several important cryptosystems make use of modular arithmetic. This is when the answer to a calculation is always in the range $0-m$ where m is the modulus.

Example: $17 \bmod 5 = 2$ $-11 \bmod 11 = 0$
 $-1 \bmod 11 = 10$ $25 \bmod 5 = 0$

$\Rightarrow$ Two numbers n and s are said to be 'congruent' $\bmod m$ if

$$n \bmod m = s \bmod m$$

Example: if $6 \bmod 5 = 1 \bmod 5$
then, $6 \equiv 1 \bmod 5$

The differences between n and s will be a multiple of m ,

So $n - s = km$ for some value of k

$$\therefore 16 - 10 = 2 * 5.$$

$\Rightarrow$ Computational complexity of calculating mod is

$$O(b^2)$$

$\Rightarrow$ Computational complexity of performing multiplication

mod m is $O(b^2)$

$\rightarrow$ computational complexity of calculating $x^n \bmod m$

is $O(b^3)$ where b is the size of n .

Modular Division:

$$a \div b \bmod c$$

$\neq$ need to multiply 'a' by inverse of $(b \bmod c)$

For inverse of $(b \bmod c)$; $(b * k) \bmod c = 1$

Example: $5 \div 3 \bmod 11$

inverse of $3 \bmod 11$ ($3 * 4 = 12 \bmod 11 = 1$)

Thus, $5 \div 3 \bmod 11$

$$\Rightarrow 5 * 4 \bmod 11$$

$$\Rightarrow 9 \bmod 11$$

It's relatively easy to find the inverse of $x \bmod m$ using Euclid's algorithm.

It's computational complexity $O(b^3)$ where b is the size of m . x doesn't have an inverse $\bmod m$ unless x and m are co-prime.

Euclid's GCD Algorithm:

Formula: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$

Example: $\text{gcd}(412, 260) = 4$

a	412	260	152	108	44	20	4
b	260	152	108	44	20	4	0

The overall running time of Euclid GCD is proportional to the number of recursive call it make.

A number $n \geq 2$ which isn't prime is called Composite.

Primality testing: Test if 139 and 143 are prime.

Relatively prime numbers: Two numbers are relatively prime if their $\text{gcd} = 1$.

Factor: A number that is multiplied by another to give product.

$7 \times 8 = 56$	108	125	250	315	0
✓ Factor	11	108	125	250	0

A number that divides evenly into another.

$$56 \div 8 = 7$$

↓
Factor

Prime Factorization: The process of finding the prime factors of a number.

hundreds chart:

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

⇒ cross - 1

⇒ cross multiple of 2

⇒ cross: multiple of 3

⇒ cross: multiple of 5

⇒ cross: multiple of 7

⇒ cross: multiple of 11

Prime numbers: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97.

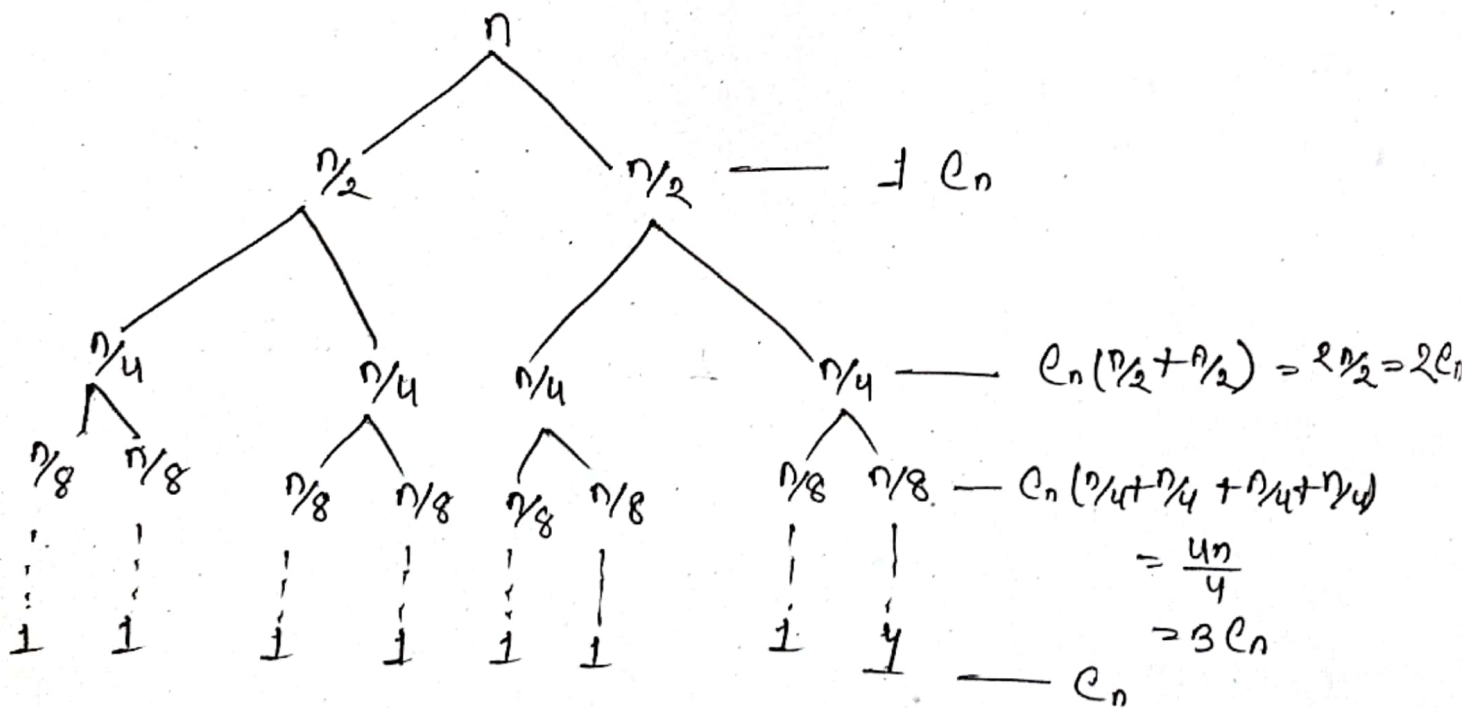
Segment-8

Recurrences and Backtracking

Recurrence: A recurrence is an equation or inequality that describes a function in terms of its value of ~~some~~ smaller inputs.

Recursion tree method:

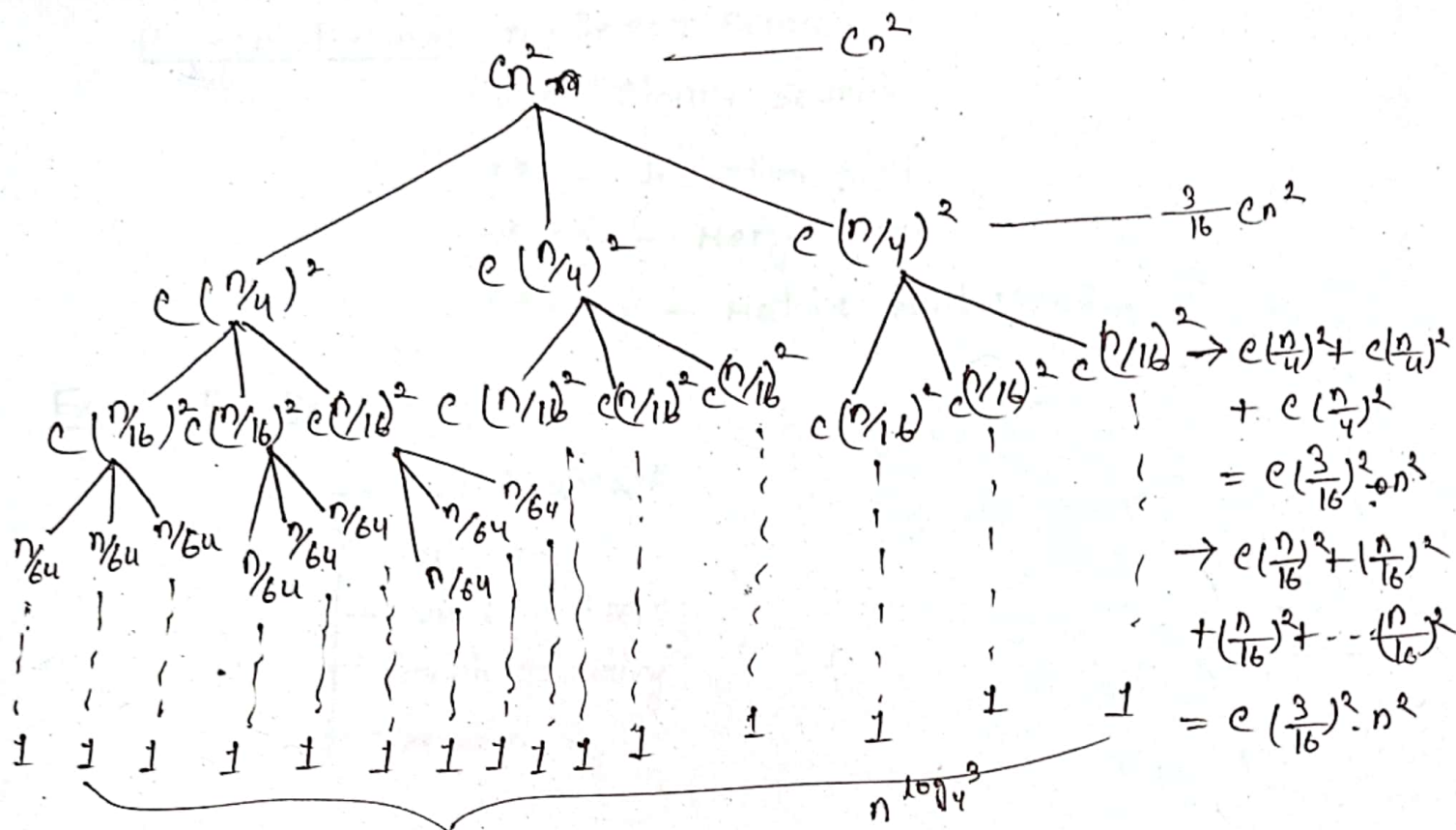
① $T(n) = 2T(n/2) + C_n$



$$T(n) = C_n \log n$$
$$= O(n \log n)$$

(2) $T(n) = 3T(n/4) + cn^2$

$T(n) = \left(\frac{n}{4}\right)T + \left(\frac{n}{4}\right)T + \left(\frac{n}{4}\right)T = (n)T$ (2)



$T(n) = cn^2 + c\left(\frac{3}{16}\right)n^2 + c\left(\frac{3}{16}\right)^2 n^2 + \dots$

$= cn^2 \left(1 + \frac{3}{16} + \left(\frac{3}{16}\right)^2 + \dots\right)$

$= cn^2 (1 + r + r^2 + \dots) \quad \left[\frac{1}{1-r}; r < 1 \right]$

$= cn^2 \left(\frac{1}{1 - \frac{3}{16}} \right)$

$= cn^2 \cdot \frac{16}{13}$

$= O(n^2)$

P, NP, NPE, NPH problem:

Polynomial time: n -linear search.

$\log n$ - Binary search

n^2 - Insertion sort

$n^2 \log n$ - Merge sort

n^3 - Matrix Multiplication

Exponential time:

2^n - 0/1 knapsack

TSP

Sum of subsets

Graph colouring

Hamilton cycle

Deterministic Algorithm: (no choice)

A deterministic algorithm is one whose behavior is completely determined by its input and the sequences of its instructions.

Non-deterministic Algorithm: (choices, success, failure)

A non-deterministic algorithm is one in which the outcome cannot be predicted with certainty even if the inputs are known.

P class: The class of decision problems that have polynomial time deterministic algorithm.

⇒ The class of problems which can be solved by a deterministic polynomial algorithm.

Example: MST

Fractional Knapsack

Single-source shortest-path

Strong Connectivity

Sorting

NP class:

The class of decision problems which can be solved in polynomial time on a non-deterministic machine or with a non-deterministic polynomial algorithm.

⇒ The class of decision problems which can be solved by a non-deterministic polynomial algorithm.

Example: Same as P

Graph Isomorphism

Hamilton cycle

3 CNF-SAT

Hamilton cycle: A hamilton cycle of an undirected graph is a simple cycle that contains every vertex.

Hamilton cycle problem in NP:

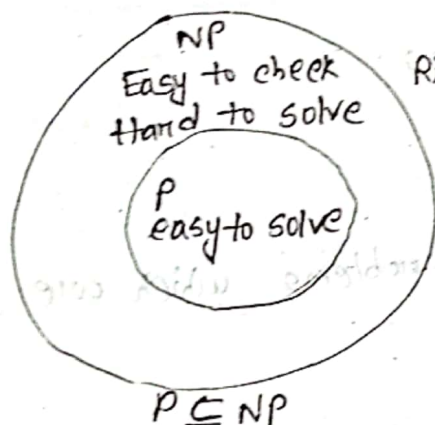
⇒ Non deterministic polynomial-time algorithm

⇒ Easy to verify solution in polynomial-time

The Satisfiability (SAT):

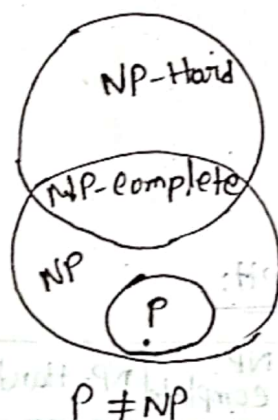
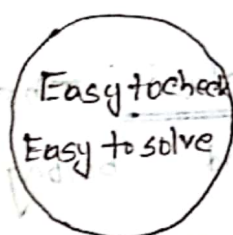
→ Given a boolean expression on n variable,
assign values such that the expression is True.

$P \subseteq NP$



Right Now

if $P = NP$



↑
complexity

P class problem: Deterministic algorithm, polynomial time

NP complete class: NP hard + Non-Deterministic Algorithm + polynomial time

NP Hard class: The class of problems to which every NP problem reduces.

⇒ If all problems $R \in NP$ are reducible to P , then P is NP hard.

Example: → Quantified Boolean Formulas
→ No Hamiltonian cycle

NP complete class: The class of problems which are NP-hard and belong to NP.

Example: → Hamilton cycle
→ 0/1 knapsack
→ Satisfiability
→ Graph and colouring
→ Maximum clique

Difference among P , NP , NPC , NPH :

	P	NP	NP-complete	NP-hard
Solvable in polynomial time	Yes			
Solution verified in polynomial time	Yes	Yes	Yes	
Reduces any NP Problem in polynomial time			Yes	Yes

Backtracking: Backtracking is the 'problem solving' method in which the solution space is scanned but with additional condition that only the possible candidate solutions are considered.

The n-Queens problem:

The problem is to place n-queens on an $n \times n$ board so that no two queens are in the same row, column or diagonal.

4-Queen Solution:

	0	1	2	3
0				
1				
2				
3				

step 1

	0	1	2	3
0	Q ₁	X	X	X
1	X	X		
2	X		X	
3	X			X

step 2

	0	1	2	3
0	Q ₁	X	X	X
1	X	X	Q ₂	X
2	X	X	X	X
3	X		X	X

step 3

	0	1	2	3
0	Q	X	X	X
1	X	X	X	Q ₂
2	X		X	X
3	X	X		X

step 4

	0	1	2	3
0	Q ₁	X	X	X
1	X	X	X	Q ₂
2	X	Q ₃	X	X
3	X	X	X	X

step 5

	0	1	2	3
0	X	Q ₁	X	X
1	X	X	X	
2		X		X
3		X		

Step 6

	0	1	2	3
0	X	Q ₁	X	X
1	X	X	X	Q ₂
2		X	X	X
3		X		X

Step 7

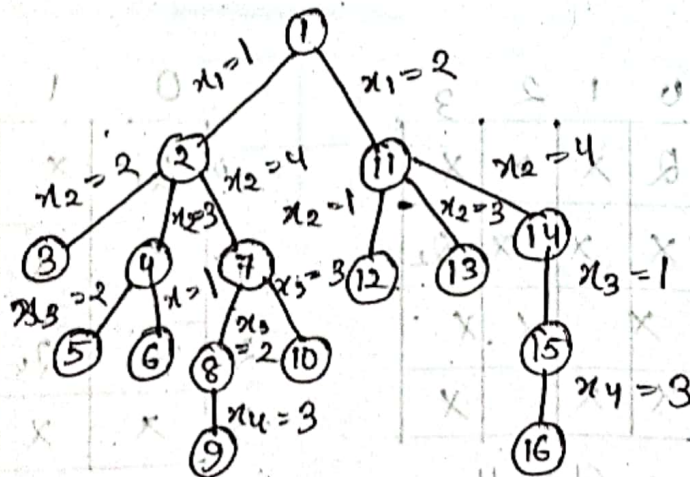
	0	1	2	3
0	X	Q ₁	X	X
1	X	X	X	Q ₂
2	Q ₃	X	X	X
3	X	X	X	X

Step 8

	0	1	2	3
0	X	Q ₁	X	X
1	X	X	X	Q ₂
2	Q ₃	X	X	X
3	X	X	Q ₄	X

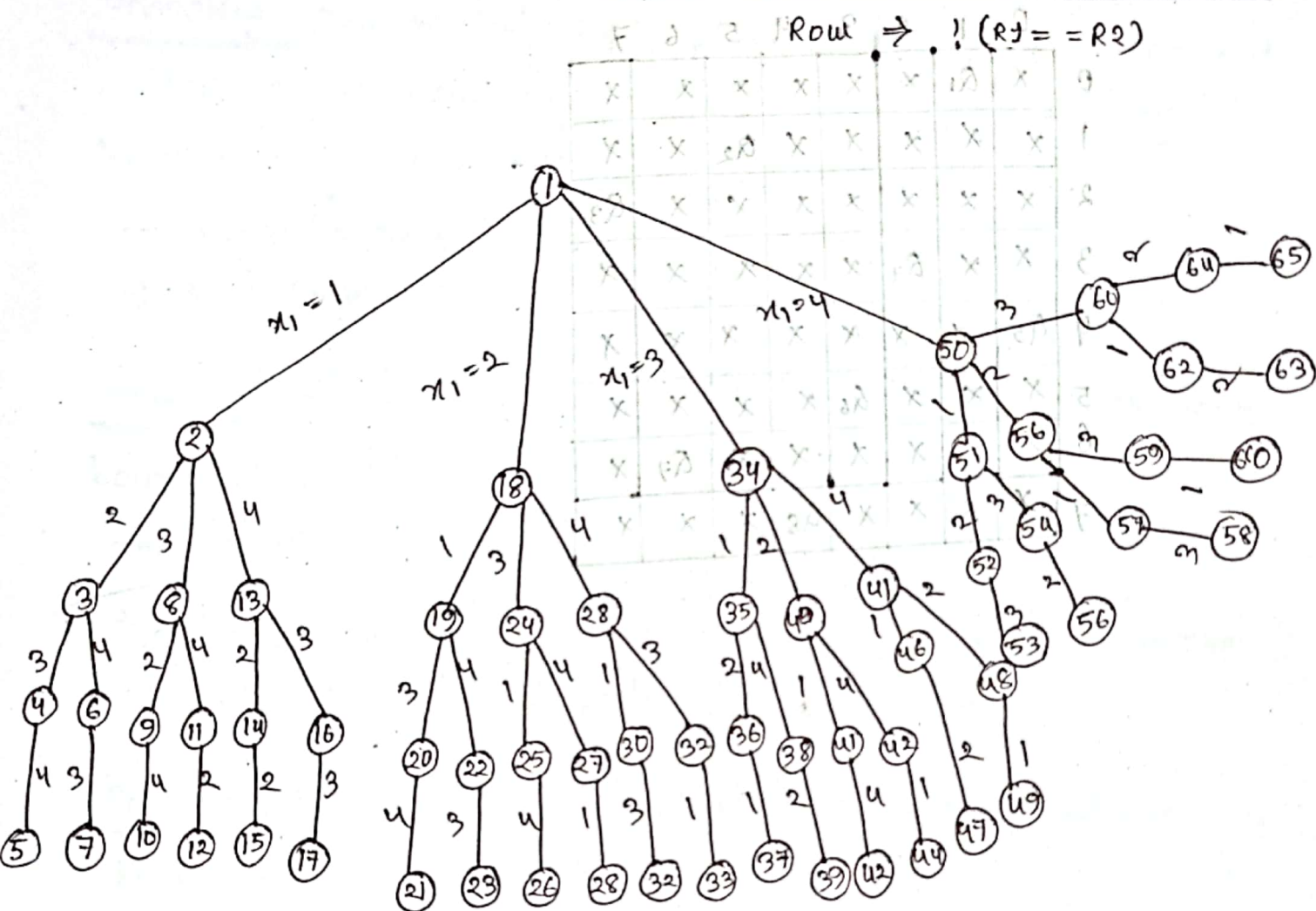
Q ₁	Q ₂	Q ₃	Q ₄
1	3	0	2

4 Queen solution during backtracking:



	0	1	2	3
0	X	X	X	X
1	X	X	X	X
2	X	X	X	X
3	X	X	X	X

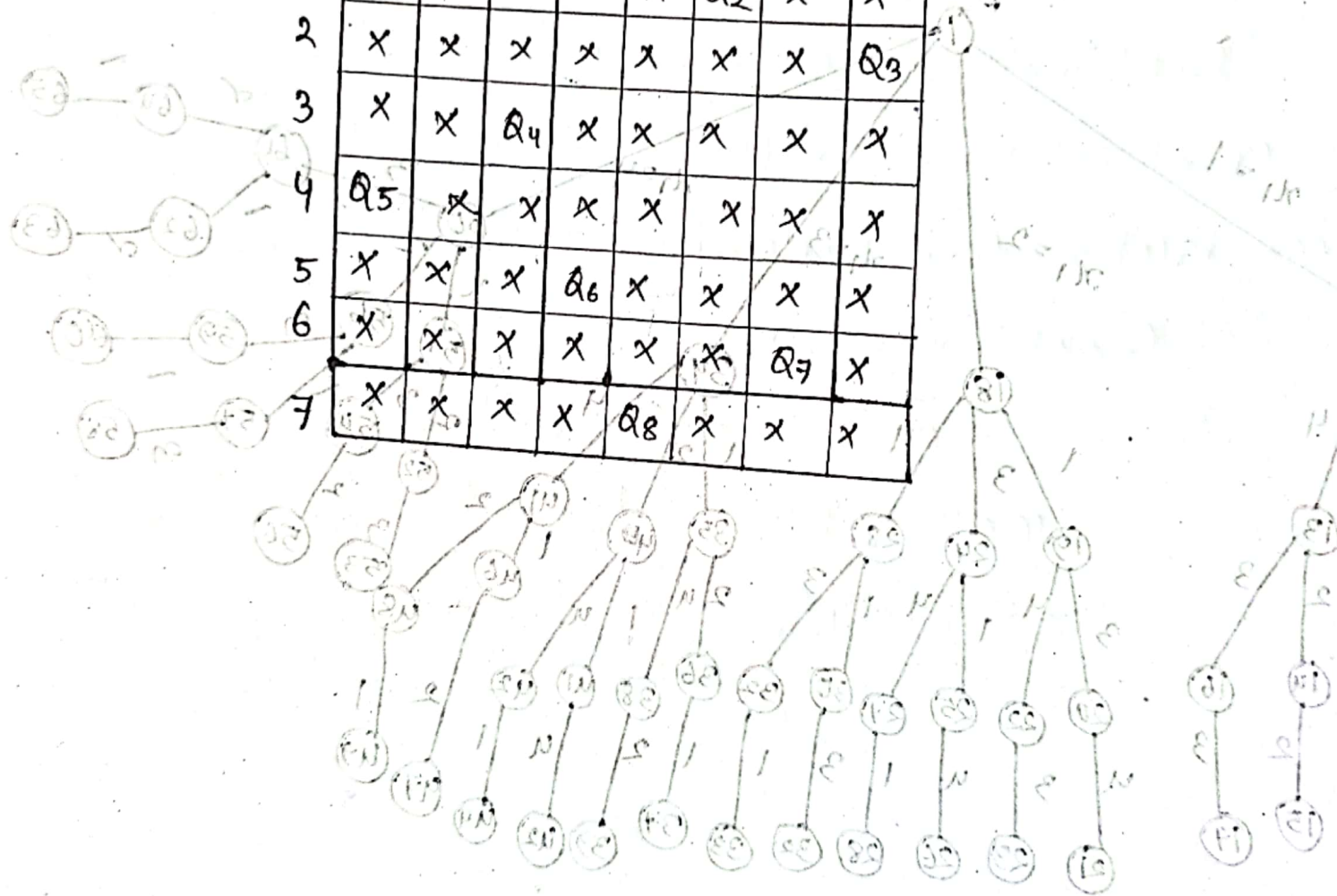
4 Queen problem space when column $\rightarrow$! (C1 = C2)



The 8 Queen Problem

(row = 0) 1 2 3 4 5 6 7

0	X	Q ₁	X	X	X	X	X	X
1	X	X	X	X	X	Q ₂	X	X
2	X	X	X	X	X	X	X	Q ₃
3	X	X	Q ₄	X	X	X	X	X
4	Q ₅	X	X	X	X	X	X	X
5	X	X	X	Q ₆	X	X	X	X
6	X	X	X	X	X	X	Q ₇	X
7	X	X	X	X	Q ₈	X	X	X



Branch and Bound Method:

Branching: The method starts with an initial problem and divides it into smaller sub problem by introducing decision Variable on constraints.

⇒ Create a tree like structure

⇒ Each node represents a sub problem

Bounding: Bounding in the context of the branch and bound method is like putting limits on boundaries on the potential solution.

⇒ It helps to figure out if a particular path in the search tree is worth pursuing or not.

Optimality Testing: The algorithm continuously updates the best known solution.

Back tracking: The algorithm may use backtracking to explore other branches if it determines that the current branch cannot lead to a better solution.

⇒ A BandB algorithm computes a number (bound) at a node to determine whether the node is promising. The number is a bound on the value of the solution that could be obtained by expanding the state space tree beyond the current node.

Application: • knapsack problem

• Integer programming

• Non linear programming

• Traveling salesman problem (TSP)

• Quadratic assignment problem (QAP)

• Maximum satisfiability problem (MAX-SAT)

• Nearest Neighbour search (NNS)

• Cutting stock problem

• False noise analysis (FNA)

• Computational phylogenetics